Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date: October 31, 2025**

Test Case ID#: STV1 **Name(s) of Testers: Joe Anderson**

Test Description:
Testing that the free function "getTiedCandidates" in src/STVElection.cc which is utilized by the STVElection class is functioning as expected. The test is stored in testing/STVElection_unittest.cc.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes___ no ___ **Function: getTiedCandidates**

Results: Pass_____ Fail_____ **Tests: stored in testing/STVElection_unittest.cc**

Preconditions for Test: A vector of candidates and the candidate piles used in the STV election have been created. This function is called when a candidate needs to be eliminated, but there could be candidates that are tied for the lowest number of votes.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Testing that the function handles one candidate properly.	Vector containing one candidate and candidate pile map containing the candidate's name with one vote	Vector containing the one candidate	Vector containing the one candidate	
2	Testing that the function handles multiple tied candidates properly	Vector containing 5 candidate and candidate pile map containing each candidate's name with one vote	Vector containing all five candidates	Vector containing all five candidates	
3	Testing that the function handles no tied candidates properly.	Vector containing 5 candidates and candidate pile map containing one candidate with 0 votes, remaining candidates all with 2 votes.	Vector containing the one candidate with 0 votes.	Vector containing the one candidate with 0 votes.	
4	Testing that the function handles a few tied candidates properly.	Vector containing 5 candidates and candidate pile map containing two tie	Vector containing the two tied candidates	Vector containing the two tied candidates	

		candidates.			

Post condition(s) for Test:

A vector of candidates that are tied with the lowest number of votes has been returned.

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date:** October 31, 2025

Test Case ID#: STV2 **Name(s) of Testers:** Joe Anderson

Test Description:
Testing that the free function "updateBallotPreferences" in src/STVElection.cc which is utilized by the STVElection class is functioning correctly. The test is stored in testing/STVElection_unittest.cc.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes___ no ___ **Function: updateBallotPreferences**

Results: Pass_____ Fail_____ **Tests: stored in testing/STVElection_unittest.cc**

Preconditions for Test: A vector of names of candidates that won/lost, ballots, a winners list, and a losers list has been created. This function is called after a round of ballot processing and winner/loser determination has finished executing.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Testing that the function transfers preference properly	Vector containing candidate name "A", vector containing a single ballot that has "A" as current preference, "B" as next preference, "C" as last preference, an empty winners list, and a losers list containing candidate name "B".	The ballot's preference has transferred to candidate "C".	The ballot's preference has transferred to candidate "C".	The preference isn't transferred to "B" because "B" is already in the losers' list.
2	Testing that the function doesn't transfer preference of a ballot that doesn't need to be updated.	Vector containing candidate name "Z", vector containing a single ballot that has "C" as current preference, an empty winners list, and a losers list containing candidate name "B".	The ballot's preference hasn't been transferred.	The ballot's preference hasn't been transferred.	The preference isn't transferred because the ballot doesn't have "Z" as its current preference.

Post condition(s) for Test:

Each ballot has updated preferences.

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: **Unit** __ **System** __ **Test Date:** **November 1, 2025**

Test Case ID#: STV3 **Name(s) of Testers: Joe Anderson**

Test Description:
Testing that the free function "findCandidateByName" in src/STVElection.cc which is utilized by the STVElection class is functioning correctly. The test is stored in testing/STVElection_unittest.cc.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: **yes** **no** **Function: findCandidateByName**

Results: **Pass** **Fail** **Tests: stored in testing/STVElection_unittest.cc**

Preconditions for Test: A list of candidates has been created.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Testing that the function finds a candidate that is in the candidate list properly	Vector containing list of candidates named "X" and "Y" and a string "X".	Function returns pointer to the candidate object with the name "X".	Function returns pointer to the candidate object with the name "X".	
2	Testing that the function returns a nullptr when it doesn't find a candidate in candidate list	Vector containing list of candidates named "X" and "Y" and a string "Z".	Function returns nullptr	Function returns nullptr	

Post condition(s) for Test:
A pointer to a candidate object has been returned if the candidate was found; a nullptr is returned otherwise.

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date: November 1, 2025**

Test Case ID#: STV4 **Name(s) of Testers: Joe Anderson**

Test Description: Testing that the public method "run" in the STVElection class is functioning correctly. The test is stored in testing/STVElection_unittest.cc.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes___ no ___ **Method: STVElection.run**

Results: Pass_____ Fail_____ **Tests: stored in testing/STVElection_unittest.cc**

Preconditions for Test: The Number of seats and whether ballots are to be shuffled or not have been determined, a vector of ballots has been created, a vector of candidates has been created, and a ballot handler object has been created.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Test 1 seat and 2 seat elections are determined correctly	3 candidates, 5 ballots (2-seat) and 3 candidates, 4 ballots (1-seat)	Winners and losers chosen correctly	Winners and losers chosen correctly	
2	Test 3 seat election with preference transfers is determined correctly	5 candidates, 10 ballots, 3 seats	Winners and losers chosen correctly	Winners and losers chosen correctly	
3	Election with tie-breakers is determined correctly	3 candidates, 3 ballots. Each candidate gets 1 vote	Winners and losers chosen correctly	Winners and losers chosen correctly	

Post condition(s) for Test:
Election results have been determined.

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date:** November 3, 2025

Test Case ID#: Candidate1 **Name(s) of Testers:** Ben Pullis

Test Description:
Testing that the candidates constructor properly Initializes the name, party, and firstVote

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes___ no ___ **Function: constructorInitialization**

Results: Pass _____ Fail_____ **Tests: stored in testing/testing.cc**

Preconditions for Test: A new candidate has been created with a name and party parameter passed in.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Testing the name gets set to the name argument	A string "testName"	The new Candidates name is equal to "testName"	The new Candidates name is equal to "testName"	
2	Testing the party gets set to the party argument	A string "testParty"	The new Candidates party is equal to "testParty"	The new Candidates party is equal to "testParty"	
3	Testing the firstVote gets set to -1	N/A	The new Candidates firstVote is equal to -1	The new Candidates firstVote is equal to -1	

Post condition(s) for Test:
The new candidates name and party attributes are set to the arguments passed and the firstVote is initialized to -1

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date:** November 3, 2025

Test Case ID#: Candidate2 **Name(s) of Testers:** Ben Pullis

Test Description:
Testing that the candidates getName method works

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes___ no ___ **Function: GetName**

Results: Pass _____ Fail_____ **Tests: stored in testing/testing.cc**

Preconditions for Test: A new candidate has been created.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Testing that getName returns a string equal to the name of the candidate	A string "testName" used when the candidate was created	The Candidates name is equal to "testName" when getName is used	The Candidates name is equal to "testName" when getName is used	

Post condition(s) for Test:
The getName method returns the correct name of the candidate in string form.

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date:** November 3, 2025

Test Case ID#: Candidate3 **Name(s) of Testers:** Ben Pullis

Test Description:
Testing that the candidates getParty method works

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes___ no ___ **Function: GetParty**

Results: Pass _____ Fail_____ **Tests: stored in testing/testing.cc**

Preconditions for Test: A new candidate has been created.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Testing that getParty returns a string equal to the party of the candidate	A string "testParty" used when the candidate was created	The Candidates party is equal to "testParty" when getParty is used	The Candidates party is equal to "testParty" when getParty is used	

Post condition(s) for Test:
The getParty method returns the correct party of the candidate in string form.

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date:** November 3, 2025

Test Case ID#: Candidate4 **Name(s) of Testers:** Ben Pullis

Test Description:
Testing that the candidates getFirstVote method works

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes no **Function: GetFirstVote**

Results: Pass Fail **Tests: stored in testing/testing.cc**

Preconditions for Test: A new candidate has been created.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Testing that getFirstVote returns an int equal to the firstVote of the candidate	A newly created candidate with an untouched firstVote	The Candidates firstVote is equal to -1	The Candidates firstVote is equal to -1	

Post condition(s) for Test:
The getFirstVote method returns the correct firstVote of the candidate as an int.

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date:** November 3, 2025

Test Case ID#: Candidate5 **Name(s) of Testers:** Ben Pullis

Test Description:
Testing that the candidates setFirstVote method works

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes no **Function: SetFirstVote**

Results: Pass Fail **Tests: stored in testing/testing.cc**

Preconditions for Test: A new candidate has been created.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Set the first vote of candidate c to 5	Int 5 representing what the new firstVote should be			
2	Check to see if the first vote was properly set to 5	N/A	Using getFirstVote, the firstVote is in fact 5	Using getFirstVote, the firstVote is in fact 5	
3	Set the first vote of candidate c to 100	Int 100 representing what the new firstVote should be			
4	Check to see if the first vote was properly set to 100	N/A	Using getFirstVote, the firstVote is in fact 100	Using getFirstVote, the firstVote is in fact 100	

Post condition(s) for Test:
The setFirstVote method sets the firstVote of the candidate to the int argument passed in.

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date:** November 3, 2025

Test Case ID#: Ballot1 **Name(s) of Testers:** Ben Pullis

Test Description:
Testing that the ballots constructor properly initializes the ballots id, rankings, and preference.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes___ no ___ **Function: ConstructorInitialization**

Results: Pass_____ Fail_____ **Tests: stored in testing/testing.cc**

Preconditions for Test: A set of rankings used to test on has been created.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Create new ballot	Use id 1 and the testing set of rankings			
2	Check to see if the id was set correctly	Id 1	The id is initialized to 1	The id is initialized to 1	
3	Check to see if the preference was set correctly	Preferences from test set of rankings	The preference is initialized to 1	The preference is initialized to 1	
4	Check to see if the size of the rankings is correct	Test set of rankings	The size of the rankings is 3	The size of the rankings is 3	
5	Check to see if the first ranking in rankings is correct	Test set of rankings	The first ranking is “c1‘	The first ranking is “c1‘	

Post condition(s) for Test:
The new ballots id, ranking, and preference all are properly initialized.

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date:** November 3, 2025

Test Case ID#: Ballot2 **Name(s) of Testers:** Ben Pullis

Test Description:
Testing that setPreference works.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes no **Function:** SetPreference

Results: Pass Fail **Tests:** stored in testing/testing.cc

Preconditions for Test: A set of rankings used to test on has been created.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Create new ballot	Use id 2 and the testing set of rankings			
2	Set a new preference	Set preference to 1	N/A	N/A	
3	Check the new preference	Current preference	Current preference is equal to 1	Current preference is equal to 1	
4	Set a new preference	Set preference to 2	N/A	N/A	
5	Check the new preference	Current preference	Current preference is equal to 2	Current preference is equal to 2	
6	Set a new preference	Set preference to current preference + 1	N/A	N/A	
7	Check the new preference	Current preference	Current preference is equal to 3	Current preference is equal to 3	

Post condition(s) for Test:
The ballots preference matches whatever it was set to using setPreference.

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date:** November 3, 2025

Test Case ID#: Ballot3 **Name(s) of Testers:** Ben Pullis

Test Description:
Testing that getCandidateRankings works.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes no **Function: GetCandidateRankings**

Results: Pass Fail **Tests: stored in testing/testing.cc**

Preconditions for Test: A set of rankings used to test on has been created.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Create new ballot	Use id 3 and the testing set of rankings			
2	Use getCandidateRankings to get the rankings	Test set of rankings	N/A	N/A	
3	Check the size of rankings	Test set of rankings	Size of the rankings is 3	Size of the rankings is 3	
4	Check the candidate at preference 1	Test set of rankings	The candidate at preference 1 is “c1”	The candidate at preference 1 is “c1”	
5	Check the candidate at preference 2	Test set of rankings	The candidate at preference 2 is “c2”	The candidate at preference 2 is “c2”	
6	Check the candidate at preference 3	Test set of rankings	The candidate at preference 3 is “c3”	The candidate at preference 3 is “c3”	

Post condition(s) for Test:
The ballot’s rankings match the rankings passed in when creating the ballot.

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date:** November 3, 2025

Test Case ID#: Ballot4 **Name(s) of Testers:** Ben Pullis

Test Description:
Testing that getId works.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes no **Function: GetId**

Results: Pass Fail **Tests: stored in testing/testing.cc**

Preconditions for Test: A set of rankings used to test on has been created.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Create new ballot	Use id 4 and the testing set of rankings			
2	Check if id is correct	A newly created ballot	The id returned from getId is 4	The id returned from getId is 4	

Post condition(s) for Test:
The ballot's id returned from getId matches the id passed in when creating the ballot.

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date: November 3, 2025**

Test Case ID#: BallotHandler_1 **Name(s) of Testers: Quang Pham**

Test Description: Verify that the BallotHandler class correctly loads candidates and ballots from multiple input files.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
testing/BallotHandler_unittest.cc

Automated: yes___ no ___

Results: Pass _____ Fail_____

Preconditions for Test: ballots1.csv and ballots2.csv exist in testing directory parallel to src and valid csv files

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Fill web output folder with ballots so BallotHandler can process	ballots1.csv, ballots2.csv	Read both files.	Reads both files.	
2	Call handler.handleFiles()	N/A	Candidates and ballots loaded	Loaded Successfully	
3	Retrieve candidates using handler.getCandidates()	N/A	Return 3 candidates Alice, Bob, Charlie	Checked each name correctly	
4	Retrieve ballots using handler.receiveBallots()	N/A	Return 3 ballots from the 2 files	Returned 3 ballots correctly	

Post condition(s) for Test:

- BallotHandler contains 3 loaded candidates and 3 ballots
- No memory leaks
- Internal data structures populated

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date: 11/3**

Test Case ID#: PluralitySystemTest1 **Name(s) of Testers: Adam Douiri**

Test Description: Does a full run-through of the system assuming a plurality election.

Automated: yes no **Indicate where are you storing the tests (what file) and the name of the method/functions being used. stored in testing/system_testing.cc**

Results: Pass Fail

Preconditions for Test: Ballot files ballots1.csv and ballots2.csv provided (in the same testing directory).

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Use fake input to open Help Window	"1\nok\nexit\n"	Help Window Text, STV Election Help Text, Help Window closed.	Actual result matched expected result.	
2	Use BallotHandler to create ballots and candidates.	ballots1.csv and ballots2.csv	Ballots and Candidates match the data provided by file.	Actual result matched expected result	
3	Run Plurality election	Ballots and Candidates from the previous step.	Alice expected to win the election.	Actual result matched expected result	
4	Check to see if Audit Report successfully created	String match to see if Alice was the winner	Proper result header provided in the file, Alice is the winner	Actual result matched expected result	

Post condition(s) for Test:

> Help Window successfully opened, STV Election Help pops up.

> BallotHandler creates ballots and candidates successfully based on ballots1.csv and ballots2.csv

> Plurality election successfully ran with Alice as winner.

> Audit report successfully generated.

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: **Unit** __ **System** __

Test Date: **November 3, 2025**

Test Case ID#: BallotHandler_2

Name(s) of Testers: Quang Pham

Test Description: Ensure shuffleBallots() changes the order but keeps the same elements

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
testing/BallotHandler_unittest.cc

Automated: **yes** **no**

Results: **Pass** **Fail**

Preconditions for Test: Ballots and ballothandler can be created

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Shuffle ballots with handle.shuffle()	3 test ballots created	Shuffled ballots same size as 3 test ballots	Same number of ballots after correct	
2	Sort ids of original and shuffled and compare	3 test ballots created	Same ballots in shuffled	Correct ballots in shuffled	
3					
4					

Post condition(s) for Test:

- Successfully shuffled ballot order and all ballots are maintained

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date: November 3, 2025**

Test Case ID#: BallotHandler_3 **Name(s) of Testers: Quang Pham**

Test Description: Destructor cleans memory safely

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
testing/BallotHandler_unittest.cc

Automated: yes___ no ___

Results: Pass_____ Fail_____

Preconditions for Test: ballots1.csv exists in testing directory parallel to src

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Load ballots to a ballot handle r and confirm	ballots1.csv	Greater than 0 ballots loaded	Greater than 0 ballots loaded correctly	
2	delete ballothandler	N/A	Safely free memory	Successfully frees memory safely	
3					
4					

Post condition(s) for Test:

- BallotHandler instance safely deleted
- No memory leaks

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date: November 3, 2025**

Test Case ID#: AuditReport_1 **Name(s) of Testers: Quang Pham**

Test Description: Ensure the report contains expected header and details.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
testing/AuditReport_unittest.cc

Automated: yes___ no ___

Results: Pass_____ Fail_____

Preconditions for Test: AuditReport accepts string vector of details

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Create details vector and generateReport()	Example data	audit_report.txt exists in the file system	Successfully created audit_report.txt	
2	Extract content and verify 'Election'and 'Date of the election' is in the header	N/A	Election and Date of the election appear in the audit_report.txt	Election and Date of the election successfully appear in the audit_report.txt	
3	Verify details passed to the report generator are in the file	N/A	Details passed in appear in audit_report.txt	Successfully verified details appear in the audit_report.txt	
4					

Post condition(s) for Test:

- An audit_report.txt file is created and the header and details are correctly loaded to the file

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: **Unit** __ **System** __ **Test Date:** **November 3, 2025**

Test Case ID#: AuditReport_2
Test Description: Ensure generateReport() handles empty details gracefully.

Name(s) of Testers: Quang Pham

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
testing/AuditReport_unittest.cc

Automated: **yes** ___ **no** ___

Results: **Pass** _____ **Fail** _____

Preconditions for Test: AuditReport accepts string vector of details

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Create audit report with empty vector details	Empty string vector	audit_report.txt is created	audit_report.txt successfully created	
2	Verify it still contains Election in the header	N/A	'Election' in the first line	Successfully verified 'Election" in first line	
3					
4					

Post condition(s) for Test:

- The audit_report.txt file is created even when there are no details passed (empty vector)

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date:** November 3, 2025

Test Case ID#: AuditReport_3 **Name(s) of Testers:** Quang Pham

Test Description: Ensure repeated calls overwrite a previous report file.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
testing/AuditReport_unittest.cc

Automated: yes ___ no ___

Results: Pass _____ Fail_____

Preconditions for Test: AuditReport accepts string vector of details

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Call generateReport twice with an AuditReport object and different details vectors	Example different string vectors	Details of the second file should show in the audit_report.txt and not from the first	Successfully has the details from just the second string vector after the second call of generateReport()	
2					
3					
4					

Post condition(s) for Test:

- Only the most recent call of generateReport() is what is stored in audit_report.txt

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date: 11/3**

Test Case ID#: PluralitySystemTest2 **Name(s) of Testers: Adam Douiri**

Test Description: Does a full run-through of the system assuming a plurality election, just with multiple candidates.

Automated: yes___ no ___ **Indicate where are you storing the tests (what file) and the name of the method/functions being used. stored in testing/system_testing.cc**

Results: Pass _____ Fail_____

Preconditions for Test: Ballot files ballots1.csv and ballots2.csv provided (in the same testing directory).

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Use fake input to open Help Window	"1\nok\nexit\n"	Help Window Text, STV Election Help Text, Help Window closed.	Actual result matched expected result.	
2	Use BallotHandler to create ballots and candidates.	ballots1.csv and ballots2.csv	Ballots and Candidates match the data provided by file.	Actual result matched expected result	
3	Run Plurality election with multiple candidates.	Ballots and Candidates from the previous step.	Alice and Bob expected to win the election.	Actual result matched expected result	
4	Check to see if Audit Report successfully created	String match to see if Alice and Bob were the winners	Proper result header provided in the file, Alice and Bob are jointly the winners.	Actual result matched expected result	

Post condition(s) for Test:

> Help Window successfully opened, STV Election Help pops up.

> BallotHandler creates ballots and candidates successfully based on ballots1.csv and ballots2.csv

> Plurality election successfully ran with Alice and Bob as winners.
> Audit report successfully generated.

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date: 11/3**

Test Case ID#: PluralitySystemTest3 **Name(s) of Testers: Adam Douiri**

Test Description: Does a full run-through of the system assuming a plurality election, just with a tie scenario.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
stored in testing/system_testing.cc

Automated: yes___ no ___

Results: Pass _____ Fail_____

Preconditions for Test: Ballot file ballots1.csv provided (in the same testing directory).

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Use fake input to open Help Window	"1\nok\nexit\n"	Help Window Text, STV Election Help Text, Help Window closed.	Actual result matched expected result.	
2	Use BallotHandler to create ballots and candidates.	ballots1.csv	Ballots and Candidates match the data provided by file.	Actual result matched expected result	
3	Run Plurality election	Ballots and Candidates from the previous step.	Tie happens, so Alice OR Bob expected to win the election.	Actual result matched expected result	
4	Check to see if Audit Report successfully created	String match to see if Alice OR Bob was the winner	Proper result header provided in the file, Alice OR Bob is the winner	Actual result matched expected result	

Post condition(s) for Test:
> Help Window successfully opened, STV Election Help pops up.
> BallotHandler creates ballots and candidates successfully based on ballots1.csv and ballots2.csv
> Plurality election successfully ran with Alice or Bob as winner.

> Audit report successfully generated.

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date: 11/3**

Test Case ID#: HelpWindowGetInstanceUnitTest
Test Description: Ensures getInstance() method properly returns a static reference to the Singleton.

Name(s) of Testers: Adam Douiri

Automated: yes___ no ___

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
stored in testing/HelpWindow_unittest.cc

Results: Pass _____ Fail_____

Preconditions for Test: N/A

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Assert that calling getInstance on HelpWindow does not cause a fatal error	N/A	No Fatal error was generated	Actual result matched expected result.	
2	Call HelpWindow::getInstance() twice and ensure that both references are equal to one another.	N/A	Both references are equal to one another, suggesting that each call to getInstance is to the same Singleton.	Actual result matched expected result	
3					
4					

Post condition(s) for Test:

> HelpWindow object successfully created.
> All calls to getInstance point to the same static reference of the Singleton.

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date: 11/3**

Test Case ID#: HelpWindowGetHelpUnitTest1
Test Description: Ensures that calling getHelp() provides the correct output for the given input.

Name(s) of Testers: Adam Douiri

Automated: yes___ no ___

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
stored in testing/HelpWindow_unittest.cc

Results: Pass _____ Fail_____

Preconditions for Test: sstream library to produce fake inputs/outputs.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Fetch an instance of the HelpWindow.	N/A	No Fatal error was generated	Actual result matched expected result.	
2	Call getHelp on the Window.	N/A	No Fatal error was generated	Actual result matched expected result	
3	Enter in computer input, capture output.	"1\nok\nexit\n"	STV Election Help Text captured, getHelp successfully exits	Actual result matched expected result	
4					

Post condition(s) for Test:
> HelpWindow object successfully created.
> Help Window opens
> STV Help appears

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __

Test Date: November 3, 2025

Test Case ID#: STV5

Name(s) of Testers: Joe Anderson

Test Description: Does a full run-through of the election process, assuming an STV Election is chosen.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
Classes tested: HelpWindow, BallotHandler, STVElection, AuditReport, Ballot, Candidate

Automated: yes ___ no ___

Results: Pass _____ Fail_____

Testing: stored in testing/system_testing.cc

Preconditions for Test: Ballot files ballots1.csv, ballots2.csv, and ballot3.csv are provided (stored in testing/ directory)

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Use fake input to open Help Window	"1\nok\nexit\n"	Help Window Text, STV Election Help Text, Help Window closed.	Actual result matched expected result.	
2	Use BallotHandler to create ballots and candidates.	Ballots10.csv, ballots11.csv	Ballot and Candidates match the data provided by file.	Actual result matched expected result	
3	Run STV election with 1 seat	ballots10.csv	Bob is elected. Alice and Charlie are not elected.	Actual result matched expected result	
4	Check if audit report generated correctly	audit_report.txt generated by AuditReport	Proper header result provided, as well as correct tracking of the transfer of preferences throughout the election	Actual result matched expected result	
5	Run STV election with 2 seats	ballots11.csv	Bob and Charlie are elected. Alice is not elected.	Actual result matched expected result	
6	Run STV election with 1 seats, with some tie-breaking scenarios throughout the process.	ballots12.csv	Alice is elected. Bob and Charlie are not elected.	Actual result matched expected result	Alice is elected since they received the earliest vote.

Post condition(s) for Test:

> Help Window successfully opened, STV Election Help pops up.
> BallotHandler creates ballots and candidates successfully based on ballots1.csv, ballots2.csv, and ballots3.csv.
> STV election successfully ran.
> Audit report successfully generated.

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date: 11/3**

Test Case ID#: HelpWindowGetHelpUnitTest2 **Name(s) of Testers: Adam Douiri**

Test Description: Ensures that calling getHelp() provides the correct output for the given input for multiple help selections.

Automated: yes___ no ___ **Indicate where are you storing the tests (what file) and the name of the method/functions being used. stored in testing/HelpWindow_unittest.cc**

Results: Pass _____ Fail_____

Preconditions for Test: sstream library to produce fake inputs/outputs.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Fetch an instance of the HelpWindow.	N/A	No Fatal error was generated	Actual result matched expected result.	
2	Call getHelp on the Window.	N/A	No Fatal error was generated	Actual result matched expected result	
3	Enter in computer input, capture output.	"1\nok\n2\nok\nexit\n"	STV AND Plurality Election Help Text captured, getHelp successfully exits.	Actual result matched expected result	
4					

Post condition(s) for Test:

> HelpWindow object successfully created.
> Help Window opens
> STV AND Plurality Help appears

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date:** 11/3

Test Case ID#: HelpWindowGetHelpUnitTest3 **Name(s) of Testers:** Adam Douiri

Test Description: Ensures that calling getHelp() exits if prompted with exit right away.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes___ no ___ **stored in testing/HelpWindow_unittest.cc**

Results: Pass _____ Fail_____

Preconditions for Test: sstream library to produce fake inputs/outputs.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Fetch an instance of the HelpWindow.	N/A	No Fatal error was generated	Actual result matched expected result.	
2	Call getHelp on the Window.	N/A	No Fatal error was generated	Actual result matched expected result	
3	Enter in computer input, capture output.	"exit\n"	getHelp successfully exits.	Actual result matched expected result	
4					

Post condition(s) for Test:

> HelpWindow object successfully created.
> Help Window opens
> Help Window exits successfully.

Project Name: Project 2: Voting System with Agile Team#11

Test Stage: Unit __ System __ **Test Date:** 11/3

Test Case ID#: PluralityElectionRunUnitTest1 **Name(s) of Testers:** Adam Douiri

Test Description: Simplest test for the plurality election with dummy candidates and dummy ballots.

Automated: yes___ no ___

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
stored in testing/PluralityElection_unittest.cc

Results: Pass _____ Fail_____

Preconditions for Test: Candidates and Ballots classes implemented properly, gtest.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Create dummy candidates.	new Candidate("John Pork", "TikTok Party"); new Candidate("Tim Cheese", "Reels Party");	Candidates were successfully created	Actual result matched expected result.	
2	Create dummy ballots.	ex: new Ballot(1, {{1, "John Pork"}}): John Pork had two votes, Tim Cheese has one	Ballots were successfully created.	Actual result matched expected result	
3	Run Plurality election with one seat up to win.	N/A	John Pork wins the election by majority first vote, there is only one winner	Actual result matched expected result	
4					

Post condition(s) for Test:
> There is a Single election winner
> John Pork is the single election winner.

Project Name: Project 2: Voting System with Agile — Team#11

Test Stage: Unit __ System __ **Test Date:** 11/3

Test Case ID#: PluralityElectionRunUnitTest2 **Name(s) of Testers:** Adam Douiri

Test Description: Tests the PluralityElection class's capability of handling multiple seats when electing someone.

Automated: yes___ no ___

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
stored in testing/PluralityElection_unittest.cc

Results: Pass _____ Fail_____

Preconditions for Test: Candidates and Ballots classes implemented properly, gtest.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Create dummy candidates.	Three candidates, John Pork, Tim Cheese and Marvin Beak	Candidates were successfully created	Actual result matched expected result.	
2	Create dummy ballots.	John Pork has 3 votes, Tim Cheese has 2 votes, and Marvin Beak has 1 vote.	Ballots were successfully created.	Actual result matched expected result	
3	Run Plurality election with two seats up to win.	N/A	John Pork wins the election as winner[0], alongside the second place winner Tim Cheese	Actual result matched expected result	
4					

Post condition(s) for Test:
> There is two election winners
> John Pork and Tim Cheese are those winners.

Test Stage: Unit __ System __ **Test Date:** 11/3

Test Case ID#: PluralityElectionRunUnitTest3 **Name(s) of Testers:** Adam Douiri

Test Description: Unit test for the run function that has a tie for one seat. Passes if either one of the tied candidates receives it (up to chance).

Automated: yes no **Indicate where are you storing the tests (what file) and the name of the method/functions being used.** stored in testing/PluralityElection_unittest.cc

Results: Pass Fail

Preconditions for Test: Candidates and Ballots classes implemented properly, gtest.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Create dummy candidates.	Three candidates, John Pork, Tim Cheese and Marvin Beak	Candidates were successfully created	Actual result matched expected result.	
2	Create dummy ballots.	John Pork has 2 votes, Tim Cheese has 2 votes, and Marvin Beak has 1 vote.	Ballots were successfully created.	Actual result matched expected result	
3	Run Plurality election with one seat up to win.	N/A	EITHER Tim Cheese or John Pork will win the election (up to chance).	Actual result matched expected result	
4					

Post condition(s) for Test:

> There is one election winner

> John Pork OR Tim Cheese is the winner.

Project Name: Project 2: Voting System With Agile **Team#11**

Test Stage: Unit __ System __ **Test Date: 12-08-25**

Test Case ID#: MVUnitTest1 **Name(s) of Testers: Ben Pullis**

Test Description: Tests the MV run function

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes no **Stored in MVElection unittest.cc**

Results: Pass Fail

Preconditions for Test: All dependencies are included, new candidates, ballots and a new election are created before running

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Create dummy candidates.	Three candidates, John Pork, Tim Cheese and Marvin Beak	Candidates were successfully created	Actual result matched expected result.	
2	Create dummy ballots.	John Pork has 3 votes, Tim Cheese has 2 votes, and Marvin Beak has 1 vote.	Ballots were successfully created.	Actual result matched expected result	
3	Run Plurality election with one seat up to win.	Used the ballots and candidates that were just created	John Pork will win the election	Actual result matched expected result	
4					

Post condition(s) for Test: All tests pass, the correct winner was selected, election info was outputted to screen and audit report file.

Project Name: Project 2: Voting System With Agile **Team#11**

Test Stage: Unit __ System __ **Test Date: 12-08-25**

Test Case ID#: MVUnitTest2 **Name(s) of Testers: Ben Pullis**

Test Description: Tests the MV run function with multiple seats

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes no **Stored in MVElection unittest.cc**

Results: Pass Fail

Preconditions for Test: All dependencies are included, new candidates, ballots and a new election are created before running

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Create dummy candidates.	Three candidates, John Pork, Tim Cheese and Marvin Beak	Candidates were successfully created	Actual result matched expected result.	
2	Create dummy ballots.	John Pork has 4 votes, Tim Cheese has 4 votes, and Marvin Beak has 3 vote.	Ballots were successfully created.	Actual result matched expected result	
3	Run Plurality election with 2 seats up to win.	Used the ballots and candidates that were just created	John Pork and Tim Cheese will win the election	Actual result matched expected result	
4					

Post condition(s) for Test: All tests pass, the correct winners were selected, election info was outputted to screen and audit report file.

Project Name: Project 2: Voting System With Agile **Team#11**

Test Stage: Unit __ System __ **Test Date: 12-08-25**

Test Case ID#: MVUnitTest3 **Name(s) of Testers: Ben Pullis**

Test Description: Tests a MV election that ends in a tie

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes___ no ___ **Stored in MVElection_unittest.cc**

Results: Pass _____ Fail_____

Preconditions for Test: All dependencies are included, new candidates, ballots and a new election are created before running

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Create dummy candidates.	Three candidates, John Pork, Tim Cheese and Marvin Beak	Candidates were successfully created	Actual result matched expected result.	
2	Create dummy ballots.	John Pork has 4 votes, Tim Cheese has 4 votes, and Marvin Beak has 3 votes.	Ballots were successfully created.	Actual result matched expected result	
3	Run Plurality election with 1 seats up to win.	Used the ballots and candidates that were just created	Either John Pork or Tim Cheese will win the election	Actual result matched expected result	
4					

Post condition(s) for Test: All tests pass, the correct winner was selected, election info was outputted to screen and audit report file.

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __

Test Date: 12-08-25

Test Case ID#: MVSystemTest1
Test Description: Tests a full run through from start to finish of a MV Election without a tie

Name(s) of Testers: Ben Pullis

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
Stored in system_testing.cc

Automated: yes___ no ___

Results: Pass _____ Fail_____

Preconditions for Test: All dependencies are included, MV ballot files ballots7.csv and ballots8.csv exist in the testing folder.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Stage test files by making a copy from the testing folder to the web/uploads folder in order to skip GUI	2 CSV files created specifically for a MV election without a tie	Files would get copied into web/uploads so they can be used for testing	Actual result matched expected result.	
2	Verify the correct candidates exist	The candidates from the CSV files used	There would be exactly 6 candidates	Actual result matched expected result.	
3	Verify the correct number of ballots exist	The ballots from the CSV files used	There would be exactly 16 ballots (2 files, 8 each)	Actual result matched expected result.	
4	Run the MV election	The ballots and candidates from the CSV files used	The election will run without failure, 1 winner, results show on screen and in audit report	Actual result matched expected result.	
5	Verify election winner	The result of getWinners() form the election	Shana will be the only winner	Actual result matched expected result.	
6	Verify audit report	Audit_report.txt generated in the src folder	All info in the report will correctly match the elections results	Actual result matched expected result.	

Post condition(s) for Test: All tests pass, the correct winner was selected, election info was outputted to screen and audit report file.

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __

Test Date: 12-08-25

Test Case ID#: MVSystemTest2
Test Description: Tests a full run through from start to finish of a MV Election with a tie

Name(s) of Testers: Ben Pullis

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
Stored in system_testing.cc

Automated: yes___ no ___

Results: Pass _____ Fail_____

Preconditions for Test: All dependencies are included, MV ballot file ballots9.csv exists in the testing folder.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Stage test file by making a copy from the testing folder to the web/uploads folder in order to skip GUI	1 CSV files created specifically for a MV election without a tie	File will get copied into web/uploads so they can be used for testing	Actual result matched expected result.	
2	Verify the correct candidates exist	The candidates from the CSV files used	There would be exactly 6 candidates	Actual result matched expected result.	
3	Verify the correct number of ballots exist	The ballots from the CSV files used	There would be exactly 8 ballots (1 files, 8 ballots)	Actual result matched expected result.	
4	Run the MV election	The ballots and candidates from the CSV file used	The election will run without failure, 1 winner that can very between 2 candidates due to tie, results show on screen and in audit report	Actual result matched expected result.	
5	Verify election winner	The result of getWinners() form the election	Shana or Joe will be the only winner	Actual result matched expected result.	
6	Verify audit report	Audit_report.txt generated in the src folder	All info in the report will correctly match the elections results	Actual result matched expected result.	

Post condition(s) for Test: All tests pass, the correct winner was selected, election info was outputted to screen and audit report file.

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date: 12/8**

Test Case ID#: Ballot_Handler4 **Name(s) of Testers: Joe Anderson, Adam Douiri**

Test Description: Tests that the BallotHandler class properly removes invalid STV Ballots

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
BallotHandler_unittest.cc, RemoveInvalidSTVBallots

Automated: yes no

Results: Pass Fail

Preconditions for Test: ballots4.csv is present in the same directory as BallotHandler_unittest.cc.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Create a BallotHandler, add ballot file to web uploads directory – call handleFiles.	ballots4.csv	Ballots successfully uploaded and processed.	Actual results matched expected result.	
2	Assert that there are only 4 valid ballots	N/A	There should be only 4 ballots, so true.	Actual results matched expected result.	
3	Assert that a thrown out ballot log was created.	STVBallotErrors{date}.csv	The file is present.	Actual results matched expected result.	
4					

Post condition(s) for Test: 4 ballots are deemed valid, 4 are removed, the STVBallotErrors file is present.

Project Name: Project 2: Voting System with Agile **Team#11**

Test Stage: Unit __ System __ **Test Date: 12/8/2025**

Test Case ID#: Ballot_Handler5
Test Description: Test that BallotHandler properly parses CSV headers

Name(s) of Testers: Joe Anderson

Indicate where are you storing the tests (what file) and the name of the method/functions being used.
BallotHandler_unittest.cc, TestHeaderParsing

Automated: yes___ no ___

Results: Pass _____ Fail_____

Preconditions for Test: ballots1.csv and ballots2.csv are in the testing directory, and have 3 lines at the top of the file containing election information

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Create a BallotHandler, add ballot file to web uploads directory – call handleFiles.	Ballots1.csv, ballots2.csv	Ballots successfully uploaded and processed.	Actual results matched expected result.	
2	Assert BallotHandler correctly parsed the election type, and getElectionType works correctly	Ballots1.csv, ballots2.csv	"pv" to represent plurality election	Actual results matched expected result.	
3	Assert BallotHandler correctly parsed the number of seats, and getNumSeats works correctly	Ballots1.csv, ballots2.csv	1	Actual results matched expected result.	
4	Verify candidates were parse correctly	Ballots1.csv, ballots2.csv	"Alice", "Bob", "Charlie"	Actual results matched expected result.	
5	Verify number of ballots is correct	Ballots1.csv, ballots2.csv	3 total ballots	Actual results matched expected result.	

Post condition(s) for Test: Election type, number of seats, and number of ballots have been determined